

Operators

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: Arithmetic Operators

Quick recap: Arithmetic operators • Integer division and modulo • Compound assignment • Increment / decrement • Precedence

Theory & Nuances

Arithmetic operators

$+$, $-$, $*$, $/$, $\%$ work the way you'd expect for basic math, but $/$ and $\%$ behave specially depending on whether they're operating on integers or decimals — that's important enough to call out on its own below.

Integer division and modulo

When both sides of $/$ are integers, the result is truncated toward zero, not rounded — $7 / 2$ gives 3, not 3.5 or 4. The decimal part is simply dropped. $\%$ (modulo) gives the remainder of a division, and only makes sense for integers — it's the standard tool for extracting digits, checking divisibility ($n \% 5 == 0$), and wrapping indices around a range.

Compound assignment

$x += 5$; is shorthand for $x = x + 5$;. The same pattern exists for $-=$, $*=$, $/=$, and $\% =$. It's not just shorter to type — it directly matches the mental model of "update this variable in place", which is exactly the running-sum pattern used constantly once loops start.

Increment / decrement

$++x$ and $x++$ both add 1 to x , but differ in what value the surrounding expression sees: $++x$ (pre-increment) increases x first and then uses the new value; $x++$ (post-increment) uses the current value first and increases x afterward. On their own on a line ($x++$;) , there's no visible difference — it only matters when the increment is embedded inside a bigger expression.

Precedence

When an expression mixes several operators, $*$ / $\%$ are evaluated before $+$ $-$, and operators of the same precedence are evaluated left to right. Parentheses $()$ always override the default order and should be used generously whenever the intended order isn't immediately obvious.

Key Points / Common Mistakes

- $7 / 2$ is 3 in integer division, not 3.5 — a very common first surprise.
- Dividing an integer by 0 crashes the program; dividing a double by 0.0 gives `inf`, not a crash.
- Without parentheses, precedence rules decide the answer — when unsure, add $()$.
- The difference between $++x$ and $x++$ only shows up when used inside a larger expression.

Classwork Problems

1. Last digit and first digit of a number

```
#include <iostream>
using namespace std;

int main() {
    int n = 5834; // a 4-digit number
```

```

    int lastDigit = n % 10;
    int firstDigit = n / 1000;

    cout << "Number      : " << n << endl;
    cout << "First digit : " << firstDigit << endl;
    cout << "Last digit  : " << lastDigit << endl;
    return 0;
}

```

2. Sum of digits of a 3-digit number

```

#include <iostream>
using namespace std;

int main() {
    int n = 573; // a 3-digit number

    int hundreds = n / 100;
    int tens = (n / 10) % 10;
    int units = n % 10;

    int sum = hundreds + tens + units;

    cout << "Number : " << n << endl;
    cout << "Sum of digits : " << sum << endl;
    return 0;
}

```

3. Evaluate a mixed expression, then verify

```

#include <iostream>
using namespace std;

int main() {
    int a = 7, b = 3, c = 2;

    // Precedence: * and / before +, evaluated left to right
    int result = a + b * c - a / b;

    cout << "Expression : a + b * c - a / b" << endl;
    cout << "Result      : " << result << endl;

    // Manual verification
    int step1 = b * c;      // 3 * 2 = 6
    int step2 = a / b;      // 7 / 3 = 2 (integer division)
    int verify = a + step1 - step2;

    cout << "Verified   : " << verify << endl;
    return 0;
}

```

Homework Problems

1. Total seconds to h:m:s

```

#include <iostream>
using namespace std;

int main() {
    int totalSeconds = 9045;

    int hours = totalSeconds / 3600;
    int minutes = (totalSeconds % 3600) / 60;
    int seconds = totalSeconds % 60;

    cout << totalSeconds << " seconds = "
         << hours << "h " << minutes << "m " << seconds << "s" << endl;
    return 0;
}

```

2. Simple interest calculator

```
#include <iostream>
using namespace std;

int main() {
    double principal = 5000.0;
    double rate = 6.5;
    double time = 3.0;

    double simpleInterest = (principal * rate * time) / 100.0;

    cout << "Principal : " << principal << endl;
    cout << "Rate      : " << rate << "%" << endl;
    cout << "Time      : " << time << " years" << endl;
    cout << "Simple Interest : " << simpleInterest << endl;
    return 0;
}
```

Lesson 2: Input with cin

Quick recap: Reading values with cin • Reading several inputs at once • Using input in calculations • Output formatting

Theory & Nuances

Reading values with cin

`cin >> variable;` reads a value typed by the user into a variable. `>>` is the extraction operator — the mirror image of `cout`'s insertion operator `<<`, pulling data out of the input stream instead of pushing it out.

Reading several inputs at once

`cin >> a >> b >> c;` chains extractions in one statement. `cin` treats spaces and newlines the same way, so the user can type values separated by spaces on one line, or press Enter between each — both work identically.

Using input in calculations

Once a value is read into a variable, it behaves exactly like any other variable of that type — it can be used directly inside arithmetic expressions without any extra conversion step.

Output formatting

By default, `cout` may print a long or unpredictable number of decimal digits for a double. `#include <iomanip>` plus `cout << fixed << setprecision(2)` forces output to exactly 2 decimal places — essential for money and measurements where 3.14159 should be shown as 3.14.

Key Points / Common Mistakes

- The variable's type decides how input is interpreted — reading a decimal into an `int` truncates it.
- `cin` doesn't validate input: typing letters where a number is expected leaves `cin` in a failed state.
- `setprecision/fixed` require `#include <iomanip>` — forgetting the include is a common miss.

Classwork Problems

1. Read two numbers, print sum/diff/product

```
#include <iostream>
using namespace std;

int main() {
    double a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;

    cout << "Sum      : " << (a + b) << endl;
```

```

        cout << "Difference: " << (a - b) << endl;
        cout << "Product    : " << (a * b) << endl;
        return 0;
    }

```

2. Read radius, print circle area

```

#include <iostream>
using namespace std;

int main() {
    double radius;
    cout << "Enter radius: ";
    cin >> radius;

    const double PI = 3.14159;
    double area = PI * radius * radius;

    cout << "Area of circle : " << area << endl;
    return 0;
}

```

3. Read 3 marks, print total and average

```

#include <iostream>
using namespace std;

int main() {
    double m1, m2, m3;
    cout << "Enter marks in 3 subjects: ";
    cin >> m1 >> m2 >> m3;

    double total = m1 + m2 + m3;
    double average = total / 3.0;

    cout << "Total    : " << total << endl;
    cout << "Average : " << average << endl;
    return 0;
}

```

Homework Problems

1. Bill = price x quantity (2 decimals)

```

#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double price;
    int quantity;

    cout << "Enter price per item: ";
    cin >> price;
    cout << "Enter quantity: ";
    cin >> quantity;

    double bill = price * quantity;

    cout << fixed << setprecision(2);
    cout << "Total bill: " << bill << endl;
    return 0;
}

```

2. Read a 3-digit number, print sum of digits

```

#include <iostream>
using namespace std;

int main() {
    int n;

```

```

    cout << "Enter a 3-digit number: ";
    cin >> n;

    int hundreds = n / 100;
    int tens = (n / 10) % 10;
    int units = n % 10;

    cout << "Sum of digits: " << (hundreds + tens + units) << endl;
    return 0;
}

```

Lesson 3: Relational & Logical Operators

Quick recap: Relational operators ==, !=, <, >, <=, >= • Logical &&, ||, ! • boolalpha; printing true/false • Truth tables

Theory & Nuances

Relational operators

==, !=, <, >, <=, >= compare two values and produce a bool result (true/false), not a number. The single most common bug in this area is confusing == (comparison) with = (assignment) — both compile, but only one asks a question; the other silently overwrites a variable.

Logical operators &&, ||, !

These combine boolean expressions. a && b is true only when both a and b are true. a || b is true when at least one of a or b is true. !a flips true to false and vice versa. C++ also short-circuits: in (a && b), if a is false, b is never even evaluated (same idea for || when a is already true) — this matters when the second condition has a side effect or could otherwise fail.

boolalpha

By default cout prints a bool as 1 or 0. std::boolalpha (cout << boolalpha;) switches this so bools print as the words true/false instead — purely for readability, it doesn't change how bools work.

Truth tables

A truth table lists every combination of true/false inputs and the resulting output for an operator. It's the systematic way to reason about a compound condition instead of guessing — write one out on paper whenever a condition with several && / || feels confusing.

Key Points / Common Mistakes

- == compares, = assigns — swapping them by accident still compiles.
- && and || short-circuit: the right-hand side may never run if the left side already decides the answer.
- Comparing two doubles with == is risky due to tiny precision errors; comparing ints is safe.

Classwork Problems

1. Print whether a > b

```

#include <iostream>
using namespace std;

int main() {
    int a = 12, b = 7;

    cout << boolalpha;
    cout << "a > b is " << (a > b) << endl;
    return 0;
}

```

2. Leap year as a boolean expression

```

#include <iostream>
using namespace std;

int main() {
    int year = 2024;

    bool isLeap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);

    cout << boolalpha;
    cout << year << " is a leap year: " << isLeap << endl;
    return 0;
}

```

3. Is a character a vowel? print the bool

```

#include <iostream>
using namespace std;

int main() {
    char ch = 'e';

    bool isVowel = (ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u' ||
                    ch=='A' || ch=='E' || ch=='I' || ch=='O' || ch=='U');

    cout << boolalpha;
    cout << "'" << ch << "' is a vowel: " << isVowel << endl;
    return 0;
}

```

Homework Problems

1. Voting eligibility as a boolean

```

#include <iostream>
using namespace std;

int main() {
    int age = 19;
    bool isIndianCitizen = true;

    bool canVote = (age >= 18) && isIndianCitizen;

    cout << boolalpha;
    cout << "Can vote: " << canVote << endl;
    return 0;
}

```

2. Print the truth tables of && and ||

```

#include <iostream>
using namespace std;

int main() {
    cout << boolalpha;

    cout << "AND (&&) Truth Table" << endl;
    cout << "true  && true  = " << (true && true) << endl;
    cout << "true  && false = " << (true && false) << endl;
    cout << "false && true  = " << (false && true) << endl;
    cout << "false && false = " << (false && false) << endl;

    cout << endl << "OR (||) Truth Table" << endl;
    cout << "true  || true  = " << (true || true) << endl;
    cout << "true  || false = " << (true || false) << endl;
    cout << "false || true  = " << (false || true) << endl;
    cout << "false || false = " << (false || false) << endl;

    return 0;
}

```
